

InfraTwin

Predictive maintenance on one PostgreSQL database.

Time-series anomaly detection, failure-cascade graph and semantic search — three capabilities that normally mean three systems.

DBTHON 2026

5,090,501 sensor readings · PostgreSQL 16 · pgvector · 73× faster than the unoptimised baseline

Three questions, three systems

Unplanned downtime costs a mid-size plant an estimated $50k per hour. The diagnosis is slow because the answers are scattered.

1

Is this machine behaving abnormally?

Needs a time-series database

Compare the last hour against this machine's own history — not a global threshold.

2

What else breaks if it fails?

Needs a graph database

Walk the dependency chain upward: motor → pump → line → plant.

3

When did this happen before, and what fixed it?

Needs a vector database

Search years of free-text maintenance reports by meaning, not keyword.

Three databases means three deployments, three backups, three security models — and the one question that matters most cannot be asked at all.

"Show me every machine downstream of the failing pump that has also had a heat-dissipation failure this month." That is one join across all three.

So we put all three in one database

PostgreSQL 16 with pgvector. The capabilities are extensions of one engine, not three services.

The conventional stack

InfluxDB / Timescale telemetry

Neo4j asset graph

Pinecone / Milvus embeddings

PostgreSQL everything else

4 deployments · 4 backups · 4 auth models
cross-domain joins written by hand in application code
no transaction spans them

InfraTwin

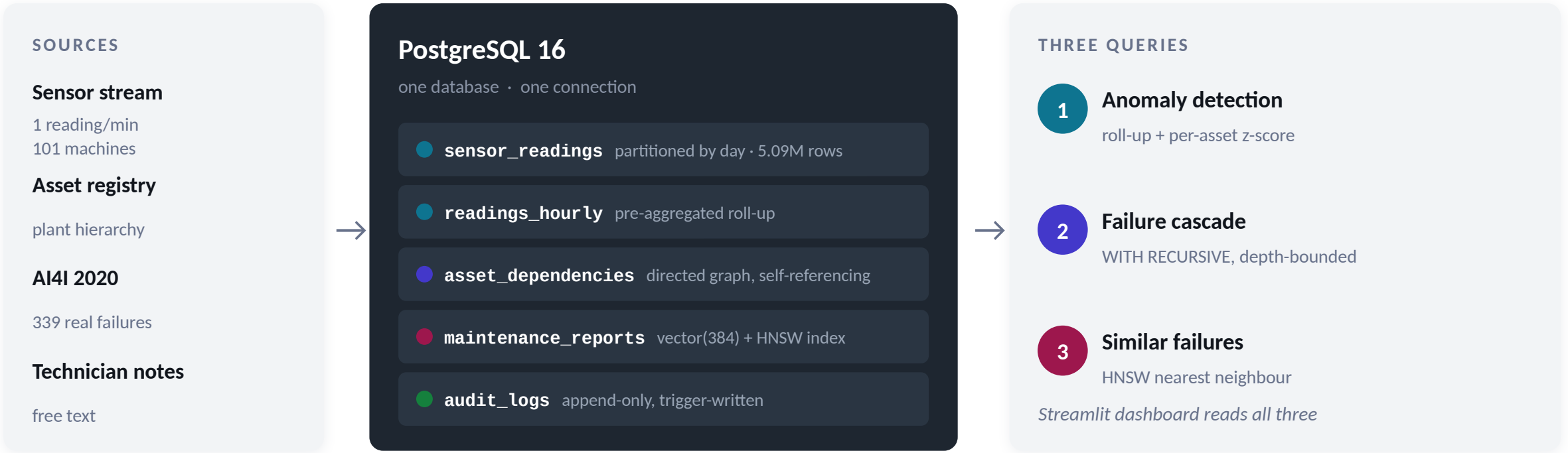
- **Partitioned tables** telemetry
- **WITH RECURSIVE** asset graph
- **pgvector + HNSW** embeddings
- **PostgreSQL** everything else

1 deployment · 1 backup · 1 security model
cross-domain joins are ordinary SQL
one ACID transaction covers all of it

Where this stops being right: a dedicated time-series engine wins past ~10^9 rows per table, and a dedicated vector database wins at billion-scale ANN. At plant scale, PostgreSQL is not the bottleneck — the integration tax is.

How it fits together

One connection. Three query patterns. The results are joinable because they share a database.



The payoff: because all three live in one database, the cascade result and the similar-failure result can be filtered by the same asset in a single statement. Across three systems that is three round trips and a join written by hand.

Technology stack

Every choice, and what it was chosen over.

Database	**PostgreSQL 16**	Everything. One engine, one backup, one security model.
Time-series	**TimescaleDB hypertable or native PARTITION BY RANGE**	Both ship. Detected at setup — the portable path needs no extension.
Vector search	**pgvector 0.8 · HNSW, cosine**	Embedding sits beside the failure record, so similarity results join directly.
Embeddings	**sentence-transformers all-MiniLM-L6-v2**	384-dim, CPU, ~80 MB. Offline lexical fallback if the model cannot be fetched.
Application	**Python 3 · psycopg 3 · pandas · numpy**	Server-side binding and native COPY for the bulk load.
Dashboard	**Streamlit 1.40**	Three panels in 190 lines instead of a hand-written frontend.
Packaging	**Docker Compose · Makefile**	make all runs the whole pipeline in about two minutes.
Testing	**pytest · 34 tests · 28 runtime self-checks**	Constraint violations are attempted and asserted to fail.
Dataset	**AI4I 2020 (UCI, CC BY 4.0)**	339 real failures across five documented modes.

Nothing here is a hosted service. The entire project runs on a laptop with Docker, or on a free hosted PostgreSQL if Docker is blocked.

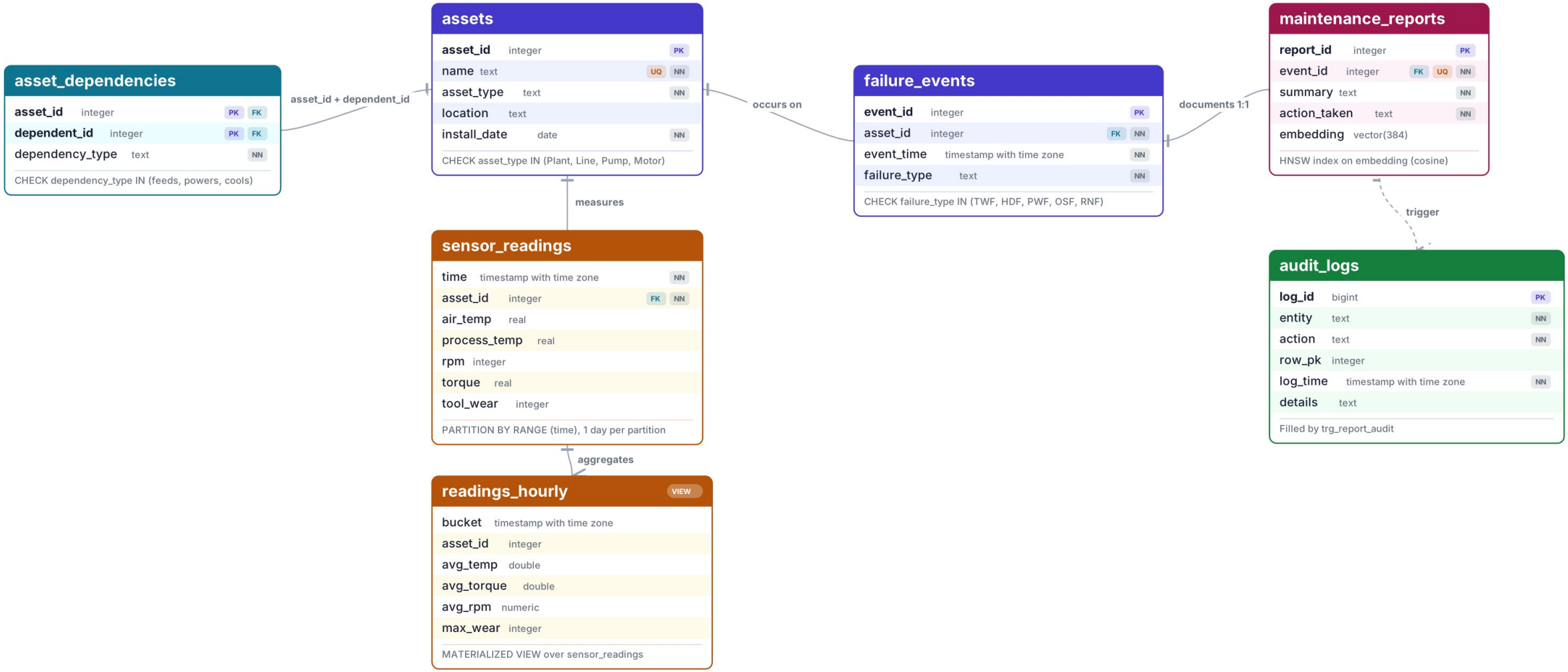


Database design

Entities, relationships, normalisation, constraints — and the decisions behind them

Database relations

Generated directly from the live pg_catalog — this diagram cannot drift from the schema. 9 tables · 1 materialized view · 17 indexes · 12 constraints.



Three design decisions worth defending

1

An edge table, not a parent_id column

asset_dependencies is a self-referencing M:N on assets.

One asset, several parents
impossible → **natural**

Typed relationship
nowhere to put it → **a column**

Add a redundant feed
schema change → **one INSERT**

A pump fed by two lines is ordinary in a real plant. parent_id cannot express it.

2

UNIQUE is what makes it 1:1

The foreign key alone would permit many reports per failure.

```
event_id INT NOT NULL UNIQUE
  REFERENCES failure_events(event_id)
```

Without UNIQUE, the similar-failure search could return three rows that are all the same incident.

3

CHECK constraints over lookup tables

asset_type and failure_type are TEXT + CHECK, not foreign keys to a lookup table.

Both are closed sets — four physical types, five AI4I codes. A lookup table buys flexibility nobody needs and costs a join on every query. If the set became user-defined, the lookup table would be correct.

Normal form: 3NF throughout. sensor_readings stores asset_id, never the machine's name or location — renaming a machine is one UPDATE, not five million. readings_hourly is the one deliberate denormalisation, and it is a measured trade.

Integrity lives in the database, not the application

Every rule holds no matter what connects — the app, psql, or a migration script.

CONSTRAINTS

assets	**UNIQUE (name)**	Two machines with the same tag
assets	**CHECK asset_type IN (…)**	Typos becoming new asset classes
asset_dependencies	**PK (asset_id, dependent_id)**	Duplicate edges
asset_dependencies	**CHECK asset_id <> dependent_id**	A machine depending on itself
failure_events	**CHECK failure_type IN (…)**	Codes outside the AI4I set
maintenance_reports	**UNIQUE (event_id)**	More than one report per failure
all foreign keys	**ON DELETE CASCADE**	Orphaned readings and edges

verify.py attempts each violation and asserts it is rejected — 28 checks, all passing. These are demonstrated, not claimed.

The audit trigger

AFTER INSERT OR UPDATE OR DELETE, FOR EACH ROW. It cannot be bypassed — it fires for the app, for psql and for a migration alike.

```
IF TG_OP = 'DELETE' THEN
  v_pk := OLD.report_id;
ELSE
  v_pk := NEW.report_id;
END IF;
```

Why the explicit branch

In a DELETE trigger NEW is unassigned. COALESCE(NEW.report_id, OLD.report_id) happens to work on PostgreSQL 16 — we tested it — but it relies on field access against an unassigned record returning NULL rather than raising. One IF removes the question.

1 Time-series layer

Partitioning for the raw stream, a pre-aggregated roll-up for the baseline.

One partition per day

```
CREATE TABLE sensor_readings (...)
  PARTITION BY RANGE (time);
```

Partition pruning. A one-hour query reads one partition, not 48 — the planner discards the rest before executing.

Cheap retention. Dropping old data is DROP TABLE — instant — instead of a DELETE that rewrites millions of rows.

readings_hourly roll-up

Hourly averages per asset, computed once, not on every dashboard load.

Storage cost	+10 MB on top of 551 MB
Freshness cost	stale until refreshed
30-day aggregate	**1,200 ms → 44 ms**

Two engines, one schema

TimescaleDB hypertable where available; native PARTITION BY RANGE where it is not. Column definitions, indexes and every downstream query are identical — only the partitioning mechanism differs. make schema detects which to use, so a locked-down lab PC or a hosted PostgreSQL with pgvector runs the same demo.

5,090,501
sensor readings loaded

52 s
to generate and load them

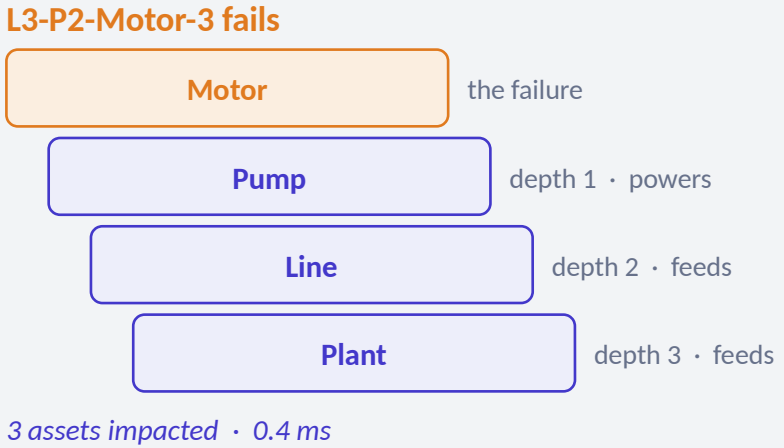
48
daily partitions

73×
faster point query

2 Graph layer

A recursive CTE walks the dependency graph upward from the failing machine.

```
WITH RECURSIVE impacted AS (
  SELECT d.dependent_id, 1 AS depth
  FROM   asset_dependencies d
  WHERE  d.asset_id = :asset
UNION ALL
  SELECT d.dependent_id, i.depth + 1
  FROM   asset_dependencies d
  JOIN   impacted i ON d.asset_id = i.asset_id
  WHERE  i.depth < 5)
SELECT a.name, a.asset_type, i.depth
FROM impacted i JOIN assets a USING (asset_id)
ORDER BY i.depth;
```



depth < 5 is not decoration

A mis-entered pair — (A→B) and (B→A) — forms a cycle, and UNION ALL without a bound never terminates. The test suite inserts exactly that cycle and asserts the query still returns.

Edge direction

`(asset_id) → (dependent_id)`

Read as “dependent_id depends on asset_id”. A motor powers a pump, so the row is (motor, pump, 'powers') and a motor failure cascades upward to the plant.

3 Vector layer

Report text becomes a vector(384). Finding a similar past failure is one ORDER BY.

```
SELECT f.failure_type, r.summary, r.action_taken,
       1 - (r.embedding <=> :qvec) AS similarity
FROM   maintenance_reports r
JOIN   failure_events f USING (event_id)
ORDER  BY r.embedding <=> :qvec
LIMIT  3;
```

<=> is cosine distance. Because the HNSW index is built on vector_cosine_ops, this is an index scan — not a scan of every report. The JOIN is the part a separate vector database cannot do.

Operator types a symptom

“temperature rising while rotational speed is low, coolant problem”

HDF sim 0.488

Process temperature reached 318.2 K, speed stayed low

HDF sim 0.479

Heat dissipation failure, coolant could not carry heat away

The failure mode worth knowing

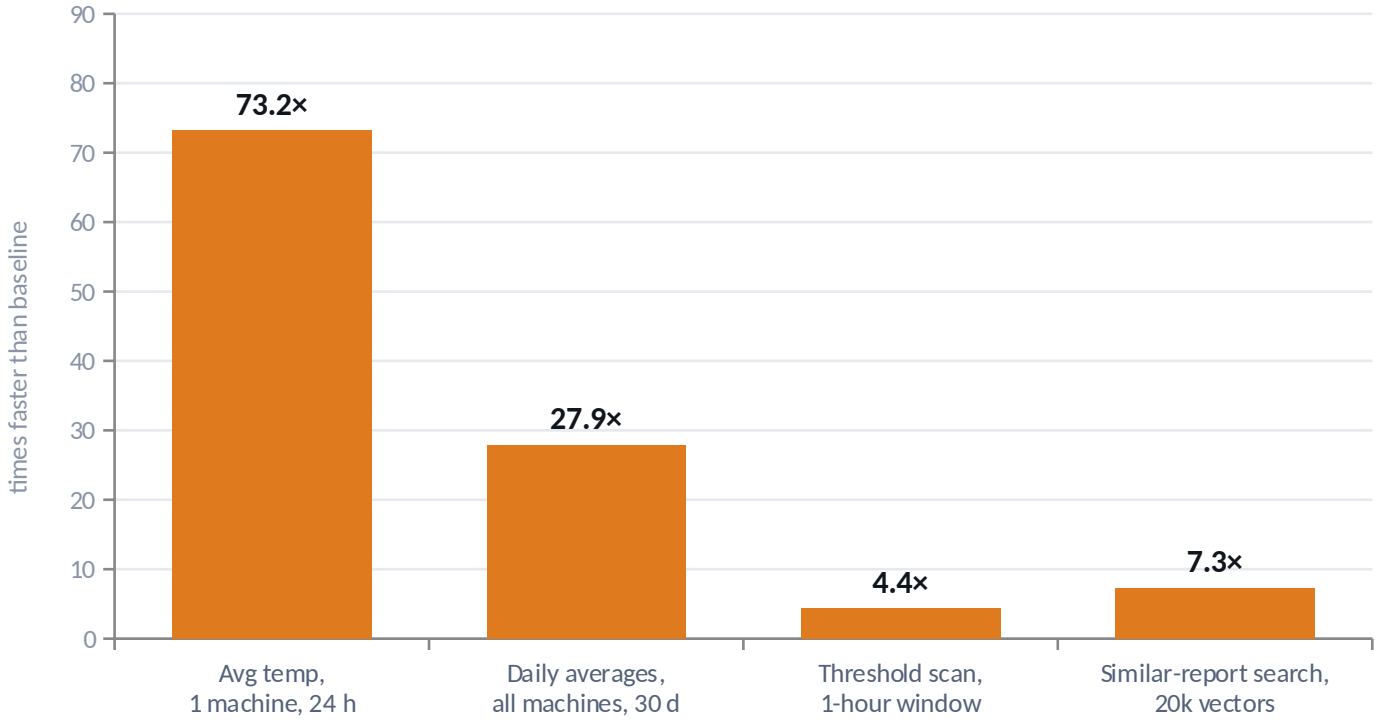
Embedding stored reports with one model and queries with another produces no error at all. Both vectors are valid, the distances compute, the results are nonsense. seed.py and app.py therefore call the same loader, and the schema's vector(384) is checked against the model's width at load time.

It works without the internet

If the model cannot be downloaded, seed.py falls back to a deterministic lexical embedder: synonym-folded bag of words, IDF-weighted, same 384 dimensions, same index, same queries. It returns the correct failure mode on all five test symptoms. The demo cannot be taken down by the venue's wifi.

Measured, not estimated

Baseline is the identical rows in one unpartitioned heap, no indexes. Median of 7 runs, 5,090,501 readings — figures read live from benchmark_results.csv.



RAW TIMINGS

	baseline	optimised
Avg temp, 1 machine, last 24h	236.3 ms	**3.23 ms**
Daily averages, all machines, 30 days	1253.2 ms	**44.92 ms**
Readings above threshold, 1-hour window	252.9 ms	**57.01 ms**
Similar-report search, 20k vectors	15.5 ms	**2.13 ms**

10 MB answers the 30-day question that would otherwise scan **551 MB**

The cost, stated plainly: the optimised table occupies 551 MB against the baseline's 293 MB. The indexes are 88% more storage. That is the deal we are making — disk is cheap, the operator waiting on a dashboard is not.

What we got wrong, and fixed

Three findings from testing against a live database rather than reasoning about it.

A z-score alone raised a false positive

On a machine whose hourly average varies by 0.1 K, a 0.3 K drift crosses z > 3. Testing surfaced exactly one such alert at z = 3.0, alongside the three real ones.

FIXED

The query now requires an absolute rise of 1.0 K as well. Three injected anomalies, three detected, no false positives.

The HNSW index is ignored on a small table

On a few hundred reports PostgreSQL picks a sequential scan — correctly, because scanning that many rows is cheaper than walking a graph index.

FIXED

The benchmark uses 20,000 vectors, where HNSW wins by the margin on the results slide. The test suite asserts the index is used there and records the sequential scan on the small table rather than hiding it.

The demo decayed as the clock advanced

The anomaly is injected relative to seed time, but the query compares windows relative to now(). After an hour the injected rows drift into the baseline they are measured against and the z-scores collapse.

FIXED

make demo reverts the previous injection exactly and re-applies it at the current time, in one transaction. Run it before presenting.

Everything in this deck was run against a live PostgreSQL 16 database. 34 tests and 28 runtime self-checks pass.

One database. Three capabilities. 73× faster.

Run it yourself: docker compose up -d · make all · make app

Where it goes next

Streaming ingestion with logical replication; alert routing from the cascade result; compression policies on chunks older than 7 days.

What it already proves

Time-series, graph and vector workloads coexist in PostgreSQL at plant scale — with one backup, one security model and joins across all three.